Complete proofs and notation reference

This companion contains complete proofs for the paper "Exact optimization of data product selection and uniform pricing under access constraints." Statements and result numbers match the current manuscript. Repeated equations retain their manuscript numbers. Equations (S1) and (S2) are proof-only identities. The notation reference restates the definitions and recurrences needed to read the proofs. The accompanying correspondence files map each item to its manuscript location.

Notation and recurrence reference

The candidate products are $\mathcal{Q} = \{1, \dots, Q\}$ with strictly increasing positive baseline values α_q. A public product menu is $(S, \mathbf{p})$, with $S \subseteq \mathcal{Q}$ and nonnegative uniform posted prices. Product q has a nonnegative fixed activation cost c_q and a per-subscription service cost s_q. Buyer type k has population n_k, valuation coefficient $\theta_k > 0$, and predetermined access limit $\bar{q}_k$. It chooses the highest-quality maximizer of $\theta_k \alpha_q - p_q$ over the accessible offered products and the outside option. A zero-utility subscription is preferred to the outside option. We set $\alpha_0 = p_0 = s_0 = 0$ for virtual product 0.

The B distinct positive access limits are $\widehat{q}_1 < \cdots < \widehat{q}_B$. The map $\beta(u)$ is the first access group permitted to purchase product u, with $\beta(Q+1) = B+1$ and $\beta(0) = 0$. The set $\mathcal{I}_b$ contains the buyer types in group b. If $B = 0$, the joint optimum is the empty menu with value 0.

$$\beta(u) = \min\{b \in \{1, \dots, B\} : u \le \widehat{q}_b\}, \tag{7}$$

Definition 4.1 (Persistent hull tree and feasibility): A persistent hull tree is an ordered tree rooted at virtual product 0, with each node's children ordered from left to right. Nonroot nodes correspond one-to-one to offered products, and each node is identified by its product index u. The edge from its parent to node u is its incoming edge, labeled by the corresponding choice threshold τ_u.

Traverse the tree in preorder, visiting each node before its subtrees in their order. Let $\nu(u)$ be the first node encountered after the entire subtree rooted at u has been visited, called the right boundary of that subtree. If no such node exists, set $\nu(u) = Q+1$. The set of access groups for which the incoming edge of node u is active is

$$J_u = \{b \in \{1, \dots, B\} : \beta(u) \le b < \beta(\nu(u))\}. \tag{8}$$

Thus, J_u contains the access groups whose choice frontiers include this edge. A PHT is feasible if and only if all the following conditions hold.

1) Product indices of nonroot nodes strictly increase in preorder.
2) All incoming edge thresholds are nonnegative.
3) Edge thresholds are nondecreasing along every path from the root to a leaf.
4) Thresholds on edges from the same parent to its children are nonincreasing from left to right.
5) The set J_u is nonempty for every nonroot node.

The population above a choice threshold and the local edge contribution are

$$W_b(t) = \sum_{k \in \mathcal{I}_b} n_k \, \mathbf{1}\{\theta_k \ge t\}, \tag{10}$$

$$g(a,u,v,t) = \big[t(\alpha_u - \alpha_a) - (s_u - s_a)\big] \cdot \sum_{b=\beta(u)}^{\beta(v)-1} W_b(t) - c_u. \tag{11}$$

Here a is the parent of u, $v = \nu(u)$ is its subtree right boundary, and t is its incoming edge threshold. The finite candidate set is

$$\mathcal{T} = \{0\} \cup \{\theta_k : 1 \le k \le K\}. \tag{13}$$

Write its sorted values as $0 = t_0 < t_1 < \cdots < t_{T-1}$. A state $H(a,b,r)$ maximizes the sum of local contributions in a possibly empty subforest attached to external parent a. Its product nodes satisfy $u > a$ and $\beta(u) < b$. Direct child thresholds are at least t_r, and the tree feasibility conditions hold. The state excludes the parent's own incoming edge and fixed cost. We use $D_b(r) = \sum_{h=1}^{b-1} W_h(t_r)$ and $z_a(r) = t_r \alpha_a - s_a$. The edge contribution and both forms of the recurrence are

$$\big[z_u(r) - z_a(r)\big]\big[D_b(r) - D_{\beta(u)}(r)\big] - c_u. \tag{14}$$

$$H(a,b,r) = \max\Big\{H(a,b,r+1), \max_{\substack{u \in \mathcal{Q},\ u>a \\ \beta(u)<b}} \big[H(a,\beta(u),r) + H(u,b,r) + \big(z_u(r) - z_a(r)\big) \cdot \big(D_b(r) - D_{\beta(u)}(r)\big) - c_u\big]\Big\}. \tag{15}$$

$$M_d(a;b,r) = \max_{\substack{u \in \mathcal{Q},\ u>a \\ \beta(u)=d}} \Big\{H(u,b,r) + z_u(r)\big[D_b(r) - D_d(r)\big] - c_u\Big\}, \quad d < b. \tag{16}$$

$$H(a,b,r) = \max\Big\{H(a,b,r+1), \max_{\max\{1,\beta(a)\} \le d < b} \big[H(a,d,r) - z_a(r)\big(D_b(r) - D_d(r)\big) + M_d(a;b,r)\big]\Big\}. \tag{17}$$

The empty subforest has value 0. Set $H(a,b,T) = 0$ and $H(a,b,r) = 0$ when $\beta(a) \ge b$; a maximum over no candidates is $-\infty$. Evaluate decreasing r, increasing b, then decreasing

a. After evaluating the current parent state, insert product a into its group cache. Algorithm 1 in the paper uses this order.

For product counts, $H_\ell(a, b, r)$ has exactly ℓ nodes, excluding a. Set $H_0 = 0$ and positive-count infeasible or terminal states to $-\infty$. Split $i + j + 1 = \ell$ when selecting last child u. For fixed-set pricing, V_0 denotes the optimum over subsets of S_0 with the fixed activation costs temporarily set to zero.

IV. Structural results

Lemma 4.1 (Removing zero-demand products): Given any feasible product menu, removing all products with no subscribers while leaving the baseline values and posted prices of the remaining products unchanged preserves the purchase decisions of every buyer type and does not reduce the provider's net profit.

Proof: None of the removed products is the final choice of any buyer under the specified choice rules. After removal, each previously chosen product and its posted price remain unchanged. Utility maximization and the same tie-breaking rules therefore lead buyers to choose the same products. Buyers who previously opted out do not switch to subscribing. Subscription revenue and service costs remain unchanged, while the provider saves the nonnegative fixed activation costs of the removed products. Net profit therefore does not decrease. □

Theorem 4.1 (Tree representation of buyer choice and price recovery): Any menu from which zero-demand products have been removed can be represented by a feasible PHT that preserves all its offered products and posted prices. Conversely, any feasible PHT yields nonnegative posted prices through

$$p_u = p_a + \tau_u(\alpha_u - \alpha_a), \qquad a = \mathrm{parent}(u), \tag{9}$$

where $\mathrm{parent}(u)$ is the parent of node u. Nonnegative edge thresholds ensure that prices are nondecreasing along each path from the root. They do not require products on different branches to be priced in quality order. For any access group b, the active edges satisfying $b \in J_u$ form a path from the root. This path represents the group's choice frontier and may contain adjacent collinear edges. Buyers of type k follow their group's path through all consecutive edges satisfying $\tau_u \le \theta_k$ and choose the last product reached. If they remain at the root, they do not subscribe.

Proof: First construct a PHT from a menu after removing zero-demand products. Insert offered products in increasing index order while maintaining the current lower convex hull sequence [1]. If the slope from the current last node to the new product is no greater than the threshold on the last node's incoming edge, remove that last node from the current sequence but retain its tree node and incoming edge. Repeat until only the root remains or the new slope exceeds the threshold on the last incoming edge. Then attach the new node to the end of the remaining sequence. This construction retains the historical branches formed under different permitted ranges.

Throughout this construction, the original menu is fixed. What changes is the prefix of offered products considered in increasing index order. Once a product is no longer chosen under the specified rules by any buyer with a positive valuation coefficient in the current prefix, extending that prefix only adds alternatives and leaves existing product utilities unchanged. The product therefore cannot be chosen again in a later prefix. Every product remaining after preprocessing has subscribers from at least one actual buyer type. It must therefore still lie on the choice frontier of the first access group permitted to purchase it. Otherwise, it could not be chosen by any later group. Its set of active access groups is consequently nonempty.

The incoming edge of node u remains active until the product at its subtree's right boundary $\nu(u)$ is inserted, or through the last access group, giving J_u. The construction ensures increasing product indices in preorder and increasing thresholds along each path. A new branch from the same parent has a threshold no greater than that of the preceding branch, which also gives the required order of sibling edge thresholds. The root price of 0, nonnegative product prices, and the order of path thresholds imply that every incoming edge threshold is nonnegative. Accumulating price increments along the edges recovers all prices in the original menu.

Conversely, given a feasible tree, the price recovery formula and nonnegative thresholds yield nonnegative posted prices. For access group b, the preorder of product indices and the access limit rule imply that the product nodes it is permitted to purchase form a prefix of the preorder traversal. If this prefix is empty, buyers can only opt out. Otherwise, let r be its last node. By the definition of J_u, an incoming edge is active exactly when node u is already in the prefix and its right boundary $\nu(u)$ is not. The active edges therefore form precisely the path from the root to r.

Consider a permitted node z outside this path. Let a be the lowest common ancestor of z and r, and let x and y be the children of a on the paths to z and r, respectively. The preorder of indices gives $a < z < y$. Together with the threshold orders along paths and among siblings, this yields

$$p_z - p_a \ge \tau_x(\alpha_z - \alpha_a) \ge \tau_y(\alpha_z - \alpha_a). \tag{S1}$$

Since $p_y - p_a = \tau_y(\alpha_y - \alpha_a)$, node z lies on or above the line segment joining a and y. Its utility is at most the corresponding convex combination of the utilities at these endpoints. If z attains the maximum utility, both a and y also attain it. The tie-breaking rule then favors the higher-quality node y. Thus, no node outside the path is chosen under the specified rules.

Along the active path, traversing edge (a, u) changes utility by $(\theta_k - \tau_u)(\alpha_u - \alpha_a)$. Because path thresholds are nondecreasing, buyers traverse all consecutive edges with thresholds no greater than θ_k, which determines their final product choice or outside option. Equality leads them onward to a higher-quality node, also implementing the rule that favors subscribing at zero utility. □

Proposition 4.1 (Exact edge-based decomposition of net profit): Any feasible PHT and its recovered posted prices

satisfy

$$\Pi(S, \mathbf{p}) = \sum_{u \in S} g\big(\text{parent}(u), u, \nu(u), \tau_u\big). \tag{12}$$

Proof: For a buyer who ultimately subscribes to product u, summing price increments along the path from the root to u gives p_u. Summing service cost increments along the same path gives s_u. The sum of incremental contributions along this path is therefore $p_u - s_u$. Multiply the path contribution of each buyer type by its population n_k and interchange the sums over buyers and edges. Each edge then contributes its incremental revenue minus service cost, multiplied by the number of buyers traversing it. Deducting the fixed activation cost c_u once for each offered product gives the stated decomposition. □

Theorem 4.2 (Finite threshold reduction): For any feasible public product menu, there exists a feasible PHT whose edge thresholds all belong to $\mathcal{T}$ and whose recovered menu preserves every buyer's purchase decision, uses no more offered products, and yields no lower provider net profit. Consequently, the joint optimization of the offered product set and uniform posted prices attains an optimum among these trees with finite candidate thresholds.

Proof: Given any feasible menu, first remove zero-demand products using Lemma 4.1. This preserves buyer choice and does not reduce net profit. If the resulting menu is empty, represent it by the tree containing only the root. Otherwise, construct its feasible PHT using Theorem 4.1.

For every nonroot node u, at least one buyer type ultimately subscribes to product u. These buyers traverse its incoming edge, so their valuation coefficient satisfies $\theta_k \geq \tau_u$. Hence, each edge threshold admits the upward rounding map

$$R(\tau) = \min\{t \in \mathcal{T} : t \geq \tau\}. \tag{S2}$$

Keep the tree structure fixed and apply the same map to every edge, replacing τ_u by $R(\tau_u)$. Since R is nondecreasing, it preserves both the nondecreasing order of path thresholds and the nonincreasing order of sibling edge thresholds. Node indices and the sets of active access groups are also unchanged. The rounded tree is therefore feasible.

For each buyer type, θ_k itself belongs to $\mathcal{T}$. If $\theta_k \geq \tau$, then θ_k is a candidate in the definition of $R(\tau)$, so $\theta_k \geq R(\tau)$. Conversely, if $\theta_k \geq R(\tau)$, then $R(\tau) \geq \tau$ implies $\theta_k \geq \tau$. Every buyer type therefore traverses exactly the same active edges before and after rounding, preserving its final product choice or decision to opt out.

No edge threshold decreases, so no product price obtained by price recovery decreases. Unchanged buyer choices preserve subscription counts and service costs. The unchanged node set preserves fixed activation costs during rounding. The provider's net profit therefore does not decrease. Together with the initial removal of zero-demand products, the entire reduction uses no more offered products and yields no lower net profit than the original menu.

Finally, the candidate product set is finite, so only finitely many ordered trees can be formed from its subsets. Each edge threshold also belongs to the finite set $\mathcal{T}$. Thus, the feasible trees with threshold labels form a finite set, including the tree containing only the root. Each tree yields a feasible menu, and any menu with continuous prices can be reduced to a tree in this set with no lower net profit. The maximum net profit over this finite set therefore equals the optimal value of the original problem and is attained by one of its trees. □

V. Algorithmic results

Lemma 5.1 (Correctness of the subforest recurrence): The recurrence computes every state $H(a, b, r)$ exactly. Representing the right boundary by an access group index and aggregating candidate roots by $\beta(u)$ both preserve the state's optimal value.

Proof: First consider the right boundary representation. Let a nonempty subforest have actual right boundary product v with $\beta(v) = b$, including the case $v = Q+1$. Let w be the last node of the subforest in preorder. Its right boundary is exactly v. A nonempty active access interval requires $\beta(w) < b$. Since β is nondecreasing in product index, all nodes u in the subforest satisfy $\beta(u) < b$. Conversely, $\beta(u) < b = \beta(v)$ implies $u < v$. Thus, group index b suffices to determine the permissible product range of the subforest. Each local edge contribution also depends on the external boundary only through b.

For any nonempty feasible subforest, suppose that the threshold on the incoming edge of the last direct child exceeds t_r. The order of sibling edge thresholds implies that all direct child edge thresholds exceed t_r, so $H(a, b, r+1)$ covers the subforest. Otherwise, let u be the last direct child, with incoming edge threshold t_r. The preceding sibling subforest has u as its right boundary and corresponds to $H(a, \beta(u), r)$. The descendant subforest of u corresponds to $H(u, b, r)$. The original subforest's net profit is the sum of the actual contributions of these two parts and the incoming edge contribution. Each part's contribution is bounded by its subproblem optimum, so the corresponding recurrence candidate is at least the net profit of the subforest.

Conversely, take optimal subforests for the two subproblems in a recurrence candidate. Every node z in the first part satisfies $\beta(z) < \beta(u)$, implying $z < u$. Every node in the second part has an index greater than u. Combining the parts therefore produces distinct nodes in increasing preorder. All direct child edge thresholds in the first part are at least t_r, so appending an edge with threshold t_r preserves the sibling order. The direct child edges of u also have thresholds at least t_r, preserving the path order. The two right boundaries are $\beta(u)$ and b, respectively, and the new edge (a, u) satisfies $\beta(u) < b$. All active access intervals are therefore nonempty. The fixed activation cost of u is deducted once in the newly added incoming edge contribution. Hence, every candidate value in the recurrence is attained by a feasible subforest.

For candidate roots with the same $\beta(u) = d$, the left state $H(a, d, r)$ and parent term $-z_a(r)[D_b(r) - D_d(r)]$ are common. Taking the maximum of the remaining terms gives exactly $M_d(a; b, r)$. Scanning a in decreasing order and inserting the current product after evaluating its state

ensures that the cache contains exactly the permitted candidate roots. Grouped computation therefore preserves the maximum candidate value.

Finally, the next threshold layer has already been computed. The left subforest depends on a smaller boundary $d < b$, and the descendant subforest depends on a larger parent product index $u > a$. The evaluation order makes every dependency available. Induction in this order, starting from the empty subforest, establishes correctness for all states. □

Theorem 5.1 (Exact joint optimization): Algorithm 1 returns a menu satisfying the buyer choice rules under access constraints and the requirement of uniform nonnegative posted prices. Its net profit equals the optimal value $H(0, B+1, 0)$ of the original joint optimization problem.

Proof: With virtual root 0, right boundary $B+1$, and a lower bound of 0 on direct child edge thresholds, the root state contains all feasible PHTs whose edge thresholds belong to $\mathcal{T}$, including the empty tree. By correctness of the subforest recurrence, this state maximizes the sum of local net profit contributions over these trees.

The tree representation of buyer choice ensures that every feasible tree yields nonnegative posted prices under which buyer choices agree with their path representation. The edge-based decomposition ensures that the recovered menu's actual net profit equals the sum of local edge contributions. Finite threshold reduction further ensures that every menu with continuous prices corresponds to a feasible tree with finite candidate thresholds and no lower net profit. The root state's optimal value therefore equals that of the original problem.

During backtracking, each recorded last subtree root and pair of states come from a transition attaining the current optimum. The product index ranges of the two parts do not overlap, so successive backtracking reconstructs a feasible tree. Recovering prices from its edge thresholds gives a menu attaining the root state's optimal value. □

Theorem 5.2 (Computational complexity): For $B \geq 1$, Algorithm 1 uses $O(Q + K\log(K+1) + QB^2T)$ arithmetic and comparison operations. Computing only the optimal value requires $O(QB + K + T)$ numerical or index storage units. Retaining menu recovery records requires $O(QBT + K + T)$ units. For explicitly encoded rational input, the algorithm has polynomial bit complexity.

Proof: Sorting buyer access limits and valuation coefficients, forming access groups, and indexing thresholds require $O(K\log(K+1))$ comparisons. Computing β in product order takes $O(Q+K)$ operations. Place each buyer type in the bucket indexed by its valuation coefficient in the candidate threshold set. As thresholds are scanned in decreasing order, add its population to the corresponding access group to maintain $W_b(t_r)$. These population updates take $O(K)$ operations in total. Computing the population prefixes $D_b(r)$ and the terms $z_a(r)$ across all layers takes $O((B+Q)T)$ operations.

The dynamic program has $O(QBT)$ states. For fixed b, r, the parent scan processes at most $Q+1$ states. Each checks at most B group candidates and performs at most one group maximum update. All state transitions therefore take $O(QB^2T)$ operations, which also covers preprocessing within threshold layers. Backtracking yields at most Q product nodes and uses at most $O(QT)$ threshold inheritance steps. Scanning products to recover prices takes $O(Q)$ operations. These costs are included in the overall bound.

Each threshold layer depends on the current and next layers. Within the current layer, states are computed in increasing access boundary and decreasing parent product index. Rolling storage therefore requires only $O(QB)$ numerical units. Group maxima, population prefixes, and product increment terms require $O(B+Q)$ units, while input buckets require $O(K+T)$ units. Menu recovery additionally stores a last subtree root or inheritance marker for each state, requiring $O(QBT)$ units.

For bit complexity, let N be the total binary encoding length of the explicit input, and let Δ be the product of all rational input denominators. Then $\log_2 \Delta \leq N$. Denominators of candidate thresholds, baseline values, and costs divide Δ. The denominator of an edge contribution, which includes a threshold multiplied by a baseline value, divides Δ^2. Buyer populations and their prefix sums are integers.

Any state value is a sum of at most Q local edge contributions, and any recovered price is a sum of at most Q edge increments. A group cache adds the corresponding product contribution to a subforest value. After expressing these quantities over the common denominator Δ^2, their numerators have bit length $O(N + \log(QK))$, polynomial in the input length. Maximum comparisons select existing candidates and preserve this bound. Every exact addition, subtraction, multiplication, and comparison can therefore be performed using a polynomial number of bit operations. Combined with the operation count, this gives polynomial bit complexity. □

Corollary 5.1 (Maximum number of offered products): For each $h \in \{0, \ldots, L\}$, the optimal net profit with at most h offered products is

$$V(h) = \max_{0 \leq \ell \leq h} H_\ell(0, B+1, 0). \tag{18}$$

For $B, L \geq 1$, one counting recurrence computes all values $V(0), \ldots, V(L)$ in $O(QB^2TL^2)$ arithmetic operations, excluding input preprocessing.

Proof: Adding $i + j + 1 = \ell$ to the decomposition at the last subtree records exactly the number of product nodes after combination. Feasibility and completeness of the original recurrence are therefore preserved. Removing zero-demand products and applying finite threshold reduction do not increase the number of offered products. Every menu satisfying the cardinality constraint thus has a feasible tree satisfying the same constraint with no lower net profit. Maximizing over node counts up to h gives the stated optimum. For each threshold layer, parent, right boundary, and access group, summing over all count splits with $\ell \leq L$ requires checking $O(L^2)$ allocations in total. There are $O(QB^2T)$ such combinations, giving the time bound.

For storage, each state additionally carries a product count in $\{0, \ldots, L\}$. Keeping two consecutive threshold layers therefore requires $O(QB(L+1))$ numerical entries. The group

cache adds at most $O(B(L+1))$ entries. Including the input buckets gives $O(QB(L+1)+K+T)$ storage units. Retaining one recovery record for each threshold layer and count gives $O(QBT(L+1)+K+T)$ units. □

Corollary 5.2 (Repricing a fixed product set): The optimal net profit for the fixed offered set S_0 is

$$V_{\mathrm{fixed}}(S_0) = V_0 - \sum_{q \in S_0} c_q. \tag{19}$$

Proof: Any price vector for the fixed set is feasible for the auxiliary problem. Its subscription revenue minus service costs is therefore at most V_0. Deducting all fixed activation costs gives the stated upper bound. The empty set attains net profit 0 directly. For nonempty S_0, let $S' \subseteq S_0$ be the set selected by the auxiliary problem. Keep the prices of S' unchanged, restore all remaining products to the offered set, and assign them the price $M = 1 + (\max_{1 \le k \le K} \theta_k)(\max_{q \in S_0} \alpha_q)$. The price M strictly exceeds every buyer's valuation of every product in S_0. All restored products therefore yield negative utility, leaving the original purchase decisions unchanged. The full set S_0 attains net profit $V_0 - \sum_{q \in S_0} c_q$, matching the upper bound.

Let $m = |S_0|$. Reindexing the products of S_0 in quality order preserves each buyer's permitted set as a prefix of length $|S_0 \cap \mathcal{Q}_k|$. If the number of distinct positive prefix lengths is $B_0 \ge 1$, the grouped recurrence has $O(mB_0T)$ states and at most B_0 group checks per state, giving $O(mB_0^2T)$ arithmetic operations apart from preprocessing. If $B_0 = 0$, no product is accessible; assigning the completion price to every product gives zero revenue and net profit $-\sum_{q \in S_0} c_q$. □

REFERENCES

[1] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," Information Processing Letters, vol. 9, no. 5, pp. 216–219, 1979.